

WEB BACKEND

PORTFOLIO

박종권

- 안정적인 트랜잭션 처리와 서버 부하 최적화 역량 보유
- 에러 로그 및 트러블슈팅 과정 문서화



안녕하세요. 저는 백엔드 개발자 박종권입니다.

비즈니스 안정성과 최적화를 책임지는 개발자입니다.

Email whdrnjs0918@naver.com
Github <https://github.com/jongkwon0918>
Velog <https://velog.io/@whdrnjs0918>

Education



K-디지털 트레이닝 JAVA 부트캠프

2025.10 - 2026.05



경남대학교 컴퓨터공학과

2020.03 - 2026.02

Certifications

- SQLD
- ADSP
- 정보처리기사

Project



ZeroRisk **Team**

KIS 모의투자 기반 주식 학습 및 대회 웹 플랫폼
2026.06 - 2026.09



JAVAJAVA **Team** **팀장**

실시간 경매 기반 중고거래 웹 서비스
2026.03 - 2026.05



CaloricCoach **Team**

AI 기반 헬스케어 앱 서비스
2024.11 - 2025.06



KIS 모의투자 기반 주식 학습 및 대회 플랫폼

Summary

- 투자 대회 도메인과 인증·보안 아키텍처를 설계하며, 자금 정합성 결함 · 반복된 보안 취약점 · 외부 API 제약을 구조적으로 해결했습니다

Action

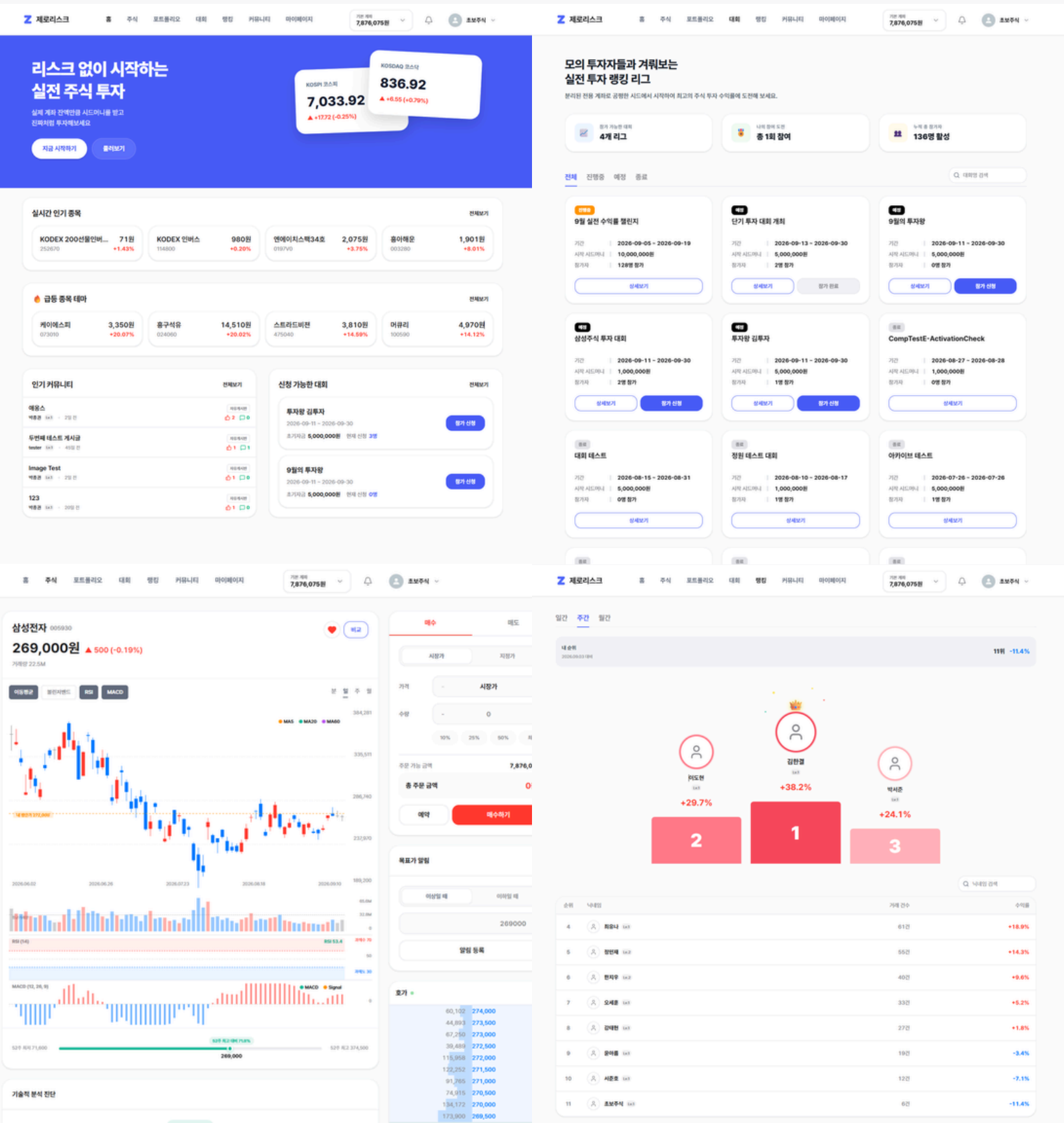
- 상금 지급 로직에 비관적 락+상태 가드 이중 적용, 중복 지급 원천 차단
- 반복된 관리자 권한 검증 누락을 계기로 URL 패턴 기반 중앙 통제로 구조 전환
- 참가자 자산 재평가 시 KIS TPS 제약을 겪고, 호출 시점 분산 · 재시도 · 중복 조회 제거까지 단계적 최적화
- OAuth2 인증 아키텍처(쿠키 기반 CSRF 방어), 로그인 Rate Limiting 등 보안 요소 구현

Period

2026.06 - 2026.09

Team

3명 (BE·FE 3)



상금 지급 동시성 결함과 권한 검증 누락,
외부 API 제약 하의 재평가 부담을
파악했습니다.

상금 중복 지급 가능성

검증 없는 상금 지급 로직

대회 종료 시 상금 지급 로직에 상태 확인·동시성 제어가 없어,
스케줄러 재실행이나 다중 서버 환경에서
중복 지급 위험이 있었습니다.

→ 이미 처리된 요청인지 확인하는 방어 자체가 부재했던 것이 원인.

상태	참가 대회명	시드머니	순위
배정	단기 투자 대회 계획	5,000,000원	-
배정	상상주식 투자 대회	1,000,000원	-
종료	정환 테스트 대회	1,000,000원	수령완료
종료	비공개 테스트 대회	3,000,000원	수령완료
종료	아카데미 테스트	5,000,000원	수령완료
종료	7월 결산지	10,000,000원	수령완료

반복되는 관리자 권한 검증 누락

세 번 반복된 같은 실수

회원 관리 · 신고/문의 · 콘텐츠 모더레이션 API에서
권한 검증 누락이 3차례 발견되어, 개별 수정이 아닌
근본 해결이 필요하다고 판단했습니다.

→ 여러 도메인이 병렬 개발되며 인증 계층 이전에 작성된 임시 코드가
통합 후에도 남아있던 것이 원인.

관리자 로그 모니터링		
관리자 페이지에서 이루어진 모든 로그 내역 및 상태 변경 로그를 실시간으로		
로그	상황 설명	수행 대상
2020-09-10 20:25:28	로그인	호환주식 → SYS1
2020-09-10 20:24:58	비밀번호	호환주식 → SYS1
2020-09-10 20:24:46	로그인	호환주식 → ANWR
2020-09-10 19:39:52	비밀번호	호환주식 → POST
2020-09-10 19:30:58	로그인	호환주식 → USER
2020-09-10 19:11:50	비밀번호	호환주식 → COM
2020-09-10 18:57:30	로그인	비밀번호 → INQUIR
2020-09-10 18:52:51	비밀번호	비밀번호 → CDP

상금 지급 동시성 결함과 권한 검증 누락,
외부 API 제약 하의 재평가 부담을
파악했습니다.

외부 API(KIS) 제약 하의 대회 재평가

TPS 제한과 중복 호출의 이중 부담

대회 종료 시 참가자 전원의 자산 재평가가 참가자 수만큼
외부 시세 API를 순차 호출하는 구조라, TPS 제한상
참가자가 많을수록 처리 시간이 늘고 일부 호출 실패 위험도 있었습니다.

→ 같은 종목을 여러 참가자가 보유해도 매번 개별 조회하던 것이 부가 원인.

순위	이름	수익률 (%)
1위	김민준	+41.1
2위	이도현	+33.1
3위	박서준	+27.1
4위	최민서	+19.4
5위	정민재	+12.7

**동시성 이중 방어와 보안 구조 전환,
외부 API 소비자 관점의 재설계를
핵심 전략으로 수립했습니다.**

상금 지급 정합성

이중 방어 전략

이미 처리된 대회인지를 확인하는 상태 가드로 순차 재실행을,
비관적 락으로 동시 실행을 막는 이중 방어 전략을 세웠습니다.
하나만으로는 서로 다른 시나리오를 놓칠 수 있다고 판단했습니다.

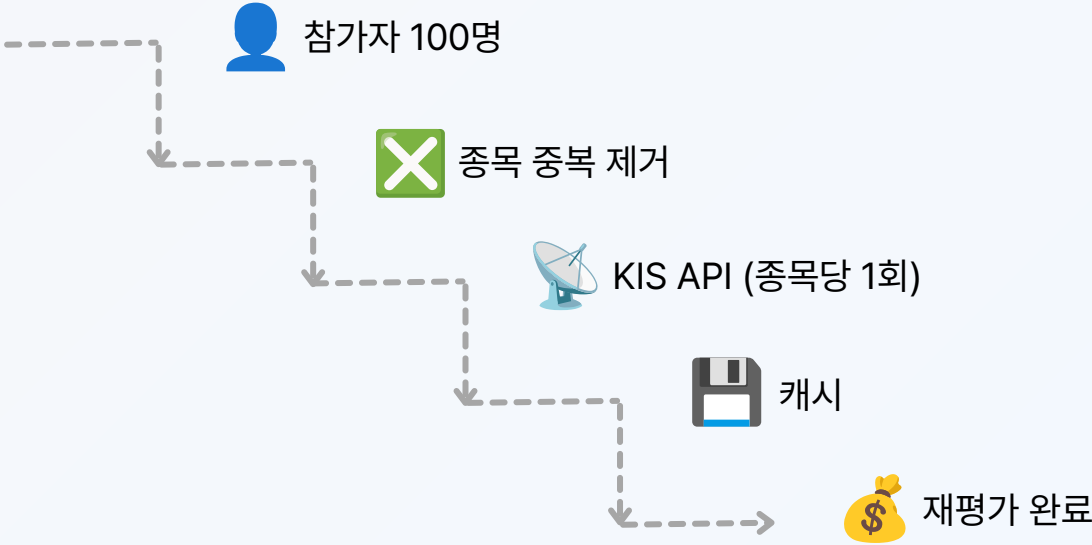
보안 결함 구조적 해소

SecurityConfig에 단일 규칙 추가

컨트롤러별 개별 검증 대신 URL 패턴 기반 전역 규칙으로 전환했습니다.
매번 주의를 기울이는 방식에서 구조적으로 강제되는 방식을 택했습니다.

동시성 이중 방어와 보안 구조 전환,
외부 API 소비자 관점의 재설계를
핵심 전략으로 수립했습니다.

KIS API 호출 흐름



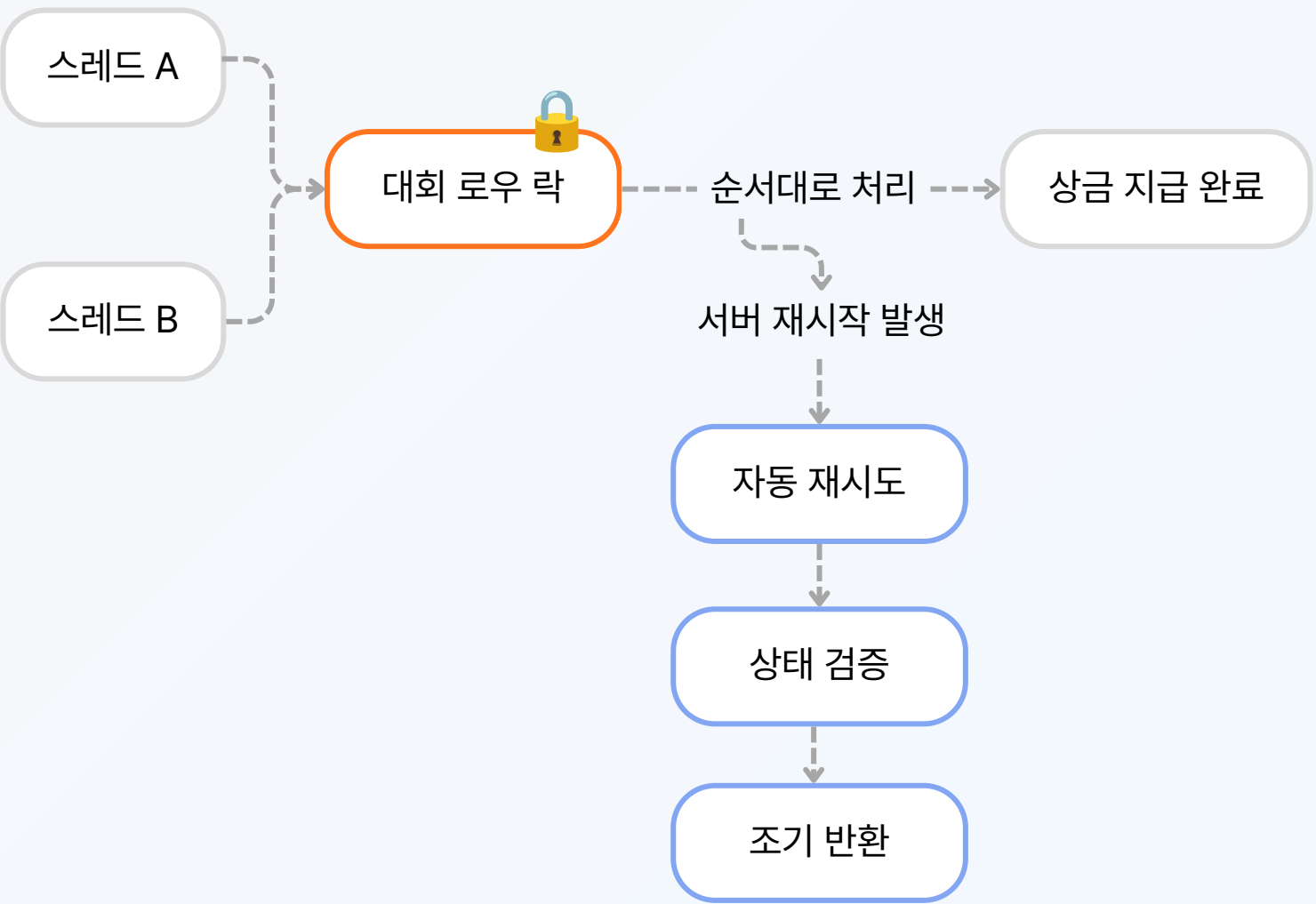
외부 API 제약

소비자 관점의 아키텍처 재설계

외부 API의 TPS 제한과 실패 가능성을 고려해 호출 시점을 분산하는 전략을 세웠습니다. 평소엔 소규모로 자주, 정확도가 중요한 종료 시점엔 전체를 한 번에 처리하며, 중복 제거와 실패 대응을 함께 설계했습니다.

대회 상금 중복 지급 방지

동시성 시나리오



상태 확인과 잠금 장치의 이중 적용

상금 지급 진입 시 대회가 이미 종료됐는지 먼저 확인해 중복 실행을 차단하고, 대회 데이터에 비관적 락을 적용해 동시 요청도 순차 처리되도록 구현했습니다. 잠그는 대상을 대회 하나로 한정해 참가자 · 상금 데이터와 얹힌 데드락 가능성을 배제했습니다.

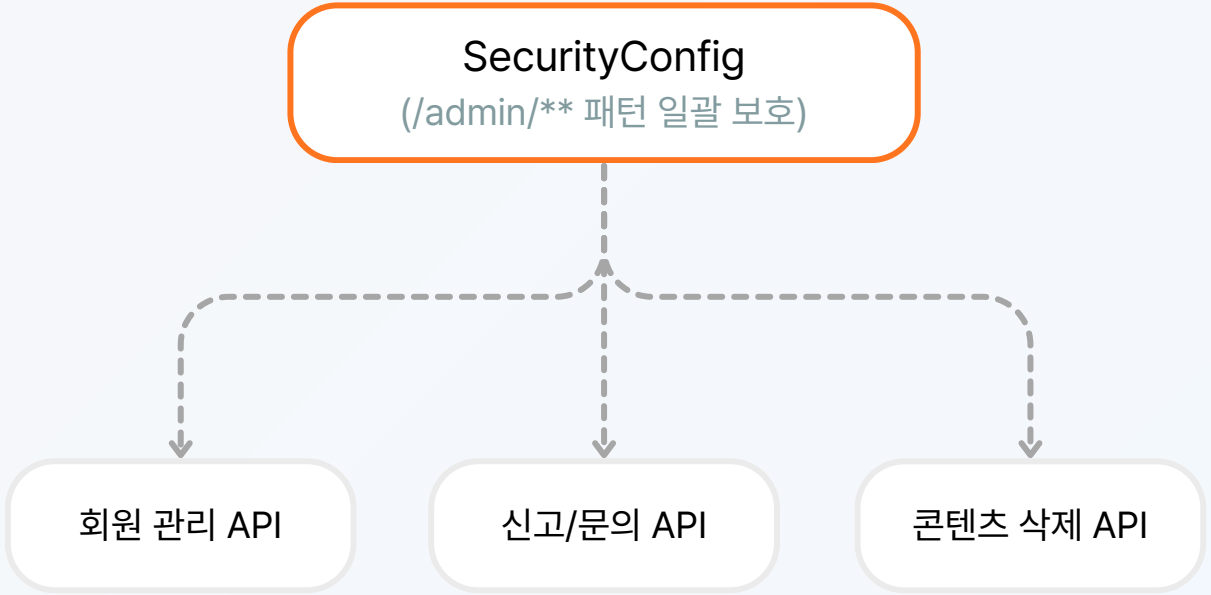
결과 서버 재시작에 의한 순차 재실행과 다중 서버 환경의 동시 실행, 양쪽 모두에서 상금 중복 지급을 원천 차단했습니다.

The screenshot shows a user dashboard for a competition. The user is 박종권 (Park Jongwon), Lv.1, with an email address jongkwon@zerorisk.com and a join date of 2026.07.19. The dashboard includes a '활동 요약' (Activity Summary) section with '대회 수상 내역' (Competition Award History) and '커뮤니티 활동' (Community Activity). Below this is a '투자 캘린더' (Investment Calendar) for September 2026. The main section is '참여한 대회' (Participated Competitions), which lists several competitions with their dates, status, prize amounts, and completion status.

대회 기간	상태	참가 대회명	시드머니	순위
2026.09.13 ~ 2026.09.30	예정	단기 투자 대회 개최	5,000,000원	-
2026.09.11 ~ 2026.09.30	예정	상상주식 투자 대회	1,000,000원	-
2026.08.10 ~ 2026.08.17	종료	장원 테스트 대회	1,000,000원	수령완료
2026.08.10 ~ 2026.08.17	종료	비공개 테스트 대회	3,000,000원	수령완료
2026.07.26 ~ 2026.07.26	종료	아카이브 테스트	5,000,000원	수령완료
2026.07.24 ~ 2026.07.31	종료	7월 캘린더	10,000,000원	수령완료

관리자 권한 검증 구조 전환

URL 패턴 기반 중앙 일괄 제어

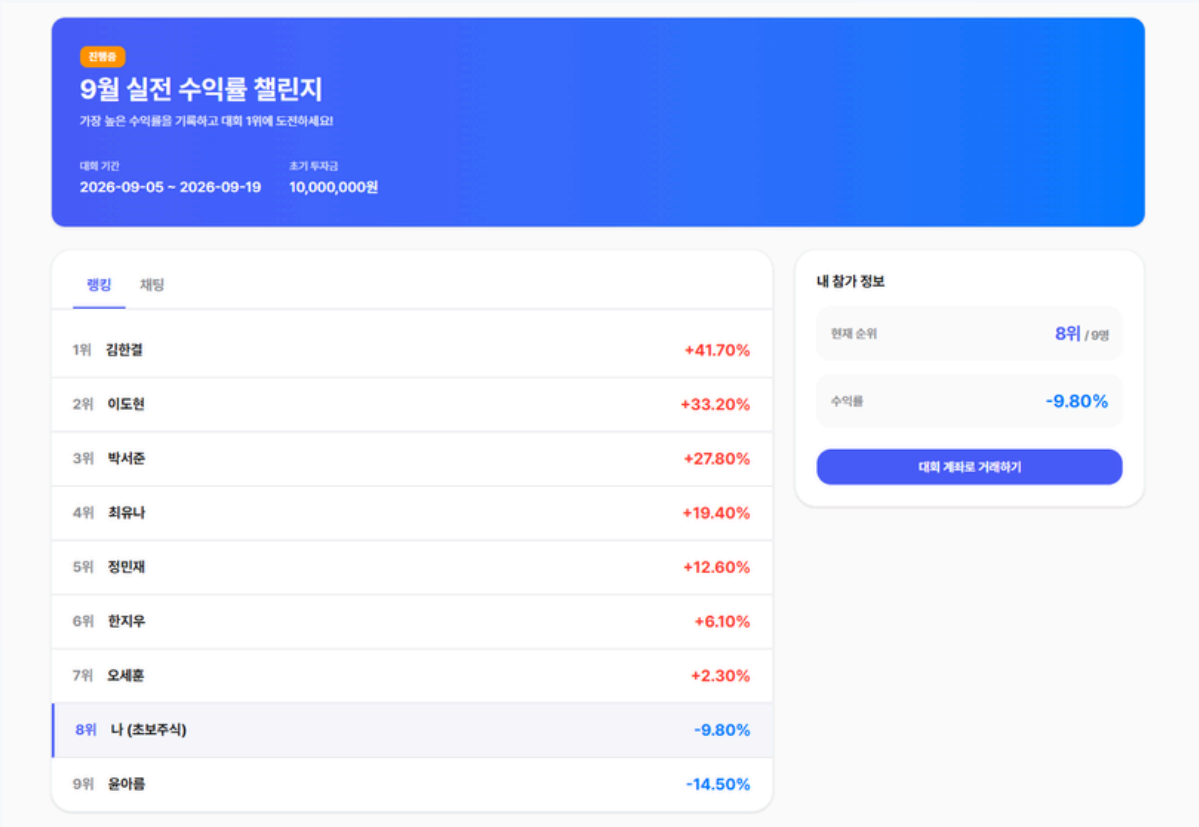


개별 어노테이션에서 URL 패턴 기반 일괄 규칙으로 전환

개별 어노테이션을 모두 제거하고, 관리자 전용 URL 패턴(/admin/**)에 규칙 하나로 통일해 권한을 검증하도록 전환했습니다. 전수 조사로 처음 예상 범위 밖에 있던 컨트롤러까지 추가로 찾아 포함시켰습니다.

결과 관리자 API가 새로 추가돼도 설정 누락 없이 일관된 보안 정책이 자동 적용됩니다.

대회 참가자 자산 재평가 최적화 (3단계 프로세스)



1. 호출 시점 분산

평소엔 체결 발생 시 해당 건만 소규모로 실시간 갱신하고, 정확도가 중요한 대회 종료 시점엔 전체 참가자를 한 번에 재평가하도록 나눠 설계했습니다.

2. 재시도와 실패 격리

외부 API 호출 실패에 대비해 지수 백오프 재시도를 적용하되, 모든 예외가 아닌 회복 가능한 일시적 장애만 선별했습니다. 최대 재시도를 넘기면 실패 기록 테이블에 저장해 유실 없이 재처리할 수 있도록 했습니다.

3. 종목 단위 중복 조회 제거

참가자별 개별 조회 대신, 전체 참가자의 보유 종목을 취합해 중복 제거 후 한 번에 조회하도록 변경했습니다. 캐시 재사용 경로가 재시도·트랜잭션을 우회하지 않도록 호출 구조도 함께 점검했습니다.

결과 외부 호출을 서로 다른 종목 수 기준으로 축소, 종목이 겹칠수록 절감 효과가 커지는 구조를 확보했습니다.

동시성 문제와 보안 결함, 외부 API 제약을
각각 다른 방식으로 해결하며
코드 기반의 아키텍처 개선을 달성했습니다.

✨ 아키텍처 개선 인사이트

- 동시성 문제는 시나리오(순차 / 동시)별로 구분해 방어해야 함을 체감.
- 반복되는 문제는 발생 자체를 구조적으로 차단하는 것이 근본 해법임을 확.
- 외부 API 제약은 인프라 흡수와 소비 로직 재설계, 두 관점이 함께 필요함을 학습.
- 재시도 · 트랜잭션(프록시 기반)은 호출 경로 전제가 깨지지 않는지 최적화 시 점검 필요.

	Before		After	성과
상금 지급 동시성	검증 없이 즉시 처리	상태 확인 + 비관적 락 이중 방어	순차/동시 시나리오 중복 지급 차단	
관리자 권한 검증	컨트롤러별 개별 검증	URL 패턴 기반 일괄 통제	신규 API도 구조적으로 자동 보호	
대회 재평가 호출 시점	단일 방식 검토	체결 시점 분산 + 종료 시 1회 전체	TPS 제약과 정확도 요구를 동시 충족	
재평가 실패 처리	재시도 로직 없음	지수 백오프 재시도 + 자체 실패 저장소	실패 격리 및 유실 없는 재처리 확보	
재평가 외부 API 호출	참가자별 개별 조회	종목 단위 중복 제거 후 일괄 조회	종목 겹침이 클수록 호출 수 대폭 감소	

JAVAJAVA



실시간 경매 기반 중고거래 웹 서비스

Summary

- 소모적인 1:1 네고 스트레스를 없애고, 입찰 경쟁을 통해 합리적인 시장가를 도출하는 플랫폼의 백엔드 아키텍처와 결제 프로세스를 구축했습니다.

Action

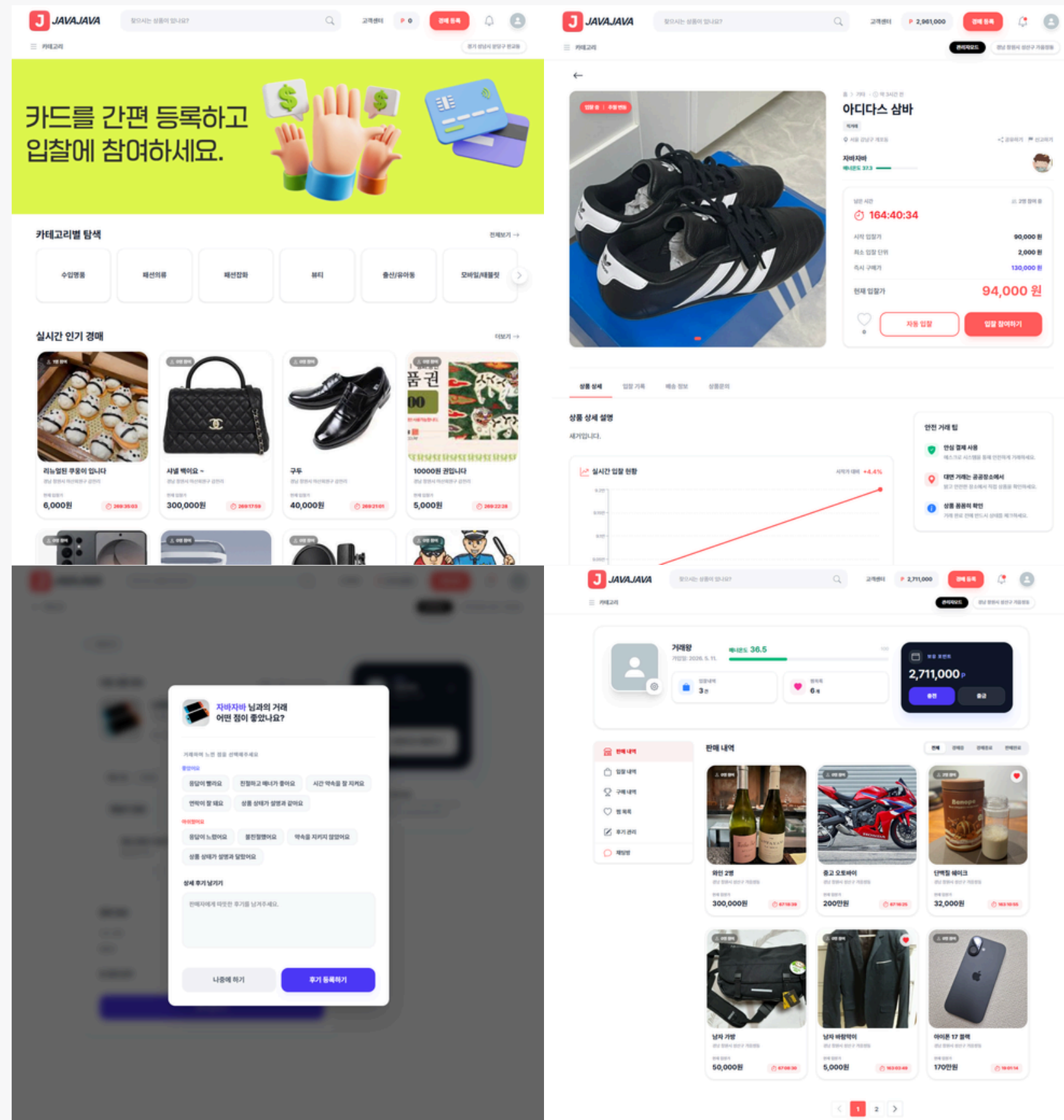
- 서버 응답 시점 기반의 메시지 처리 로직 개선으로 1:1 채팅 동기화 오류 해결
- PortOne API와 포인트 적립 로직을 단일 트랜잭션으로 묶어 결제 무결성 보장
- GitHub Actions와 Docker Compose를 활용한 CI/CD 파이프라인 구축
- 리뷰 시스템 DB 정규화 및 다대다 태그 매핑 구현
- ID 설계 규칙을 활용한 카테고리 범위 단일 쿼리 최적화
- 스왑 메모리 최적화를 통한 AWS EC2 Docker 기반 운영 DB 독립 구축

Period

2026.03 - 2026.05

Team

4명 (BE·FE 3, QA1)



결제 데이터 정합성 결함과
DB 설계 비효율, 배포 환경에서의
실시간 통신 장애를 파악했습니다.

결제 데이터 정합성 결함

결제 잔액 불일치

충전 완료 화면은 뜨지만 포인트 미반영 또는 카드 미결제가 발생.

→ 결제 요청(PortOne)과 포인트 적립이 분리 처리되면서 중간
실패 시 한쪽만 처리되는 구조적 결함이 원인.



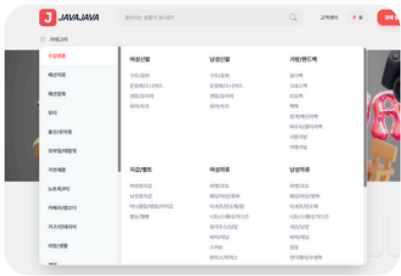
DB 설계 결함 및 쿼리 비효율

비정규화 구조

거래 후기 태그를 단일 컬럼에 쉼표 구분 문자열로 저장해
대상별 태그 집계와 데이터 가공이 불가능한 상태.

재귀 쿼리 부하

카테고리 필터링 시 대분류 선택만으로 하위 카테고리 수집을 위한
DB 조회가 최대 4회 반복 발생.



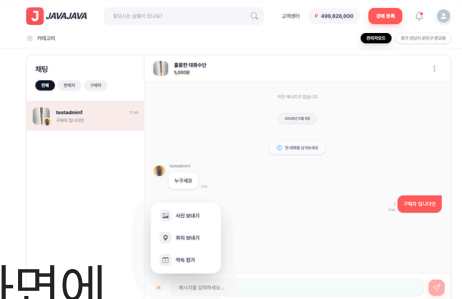
결제 데이터 정합성 결함과
DB 설계 비효율, 배포 환경에서의
실시간 통신 장애를 파악했습니다.

실시간 채팅 메시지 정합성 결함

낙관적 UI 무한 실패

메시지 전송 후 서버 저장·상대방 수신은 정상인데, 발신자 화면에서만 5초 후 '재전송(FAILED)'이 반복되는 현상 발생.

→ 중복 방지용 **UUID**의 등록 시점 오류로, 서버 응답이 도착해도 '이미 처리된 메시지'로 무시되는 논리적 결함이 원인.



결제 데이터 정합성 결함과
DB 설계 비효율, 배포 환경에서의
실시간 통신 장애를 파악했습니다.

운영 DB 서버 환경 종속성

자체 운영 DB 구축

외부 공유 서버의 Oracle DB에 의존하는 구조로,
안정적인 시연과 운영을 위해 자체 DB 서버 환경 구축 필요성 인식.

배포 환경 실시간 통신 장애

WebSocket 연결 실패

로컬에서 동작하던 채팅 기능이 배포 환경에서 405 에러 반환.

→ Nginx가 WebSocket 핸드셰이크 헤더를 전달하지 않은 것이 원인.

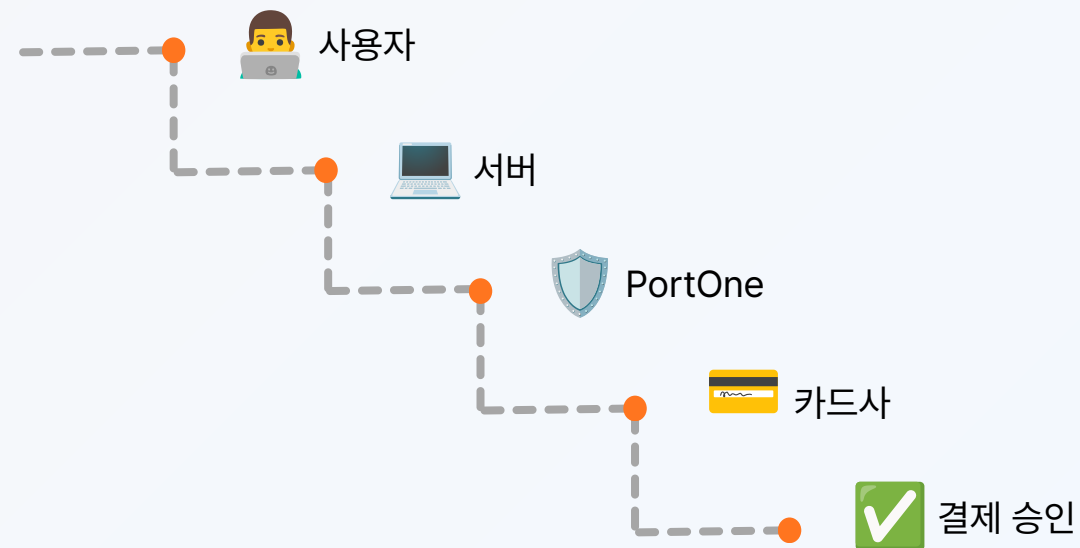
SSE 알림 지연

실시간 알림이 배포 서버에서 수 초씩 지연되거나 뭉쳐서 전달.

→ Nginx 기본 응답 버퍼링이 원인.

결제 원자성 보장과 DB 구조 개선,
배포 자동화 및 실시간 통신 안정화를
핵심 전략으로 수립했습니다.

작업 절차



트랜잭션 원자성 보장

결제 잔액 불일치

외부 결제 API 승인, 금액 검증, 내부 포인트 적립을 단일 트랜잭션으로 묶어
부분 실패 시 전체 롤백되는 구조로 설계.

DB 정규화 및 쿼리 구조 개선

비정규화 구조 재귀 쿼리 부하

다중값 컬럼을 태그 정의 테이블과 연결 테이블로 분리해 정규화 수행. 카테
고리 ID 설계 규칙을 활용해 재귀 조회를 범위 조건 단일 쿼리로 전환.

결제 원자성 보장과 DB 구조 개선,
배포 자동화 및 실시간 통신 안정화를
핵심 전략으로 수립했습니다.

WebSocket+STOMP 기반 실시간 채팅 아키텍처 설계

낙관적 UI 무한 실패

WebSocket으로 양방향 실시간 통신을 확보하고,
STOMP 프로토콜로 채팅방별 구독/발행 구조를 설계.
Save-then-Broadcast 패턴으로 메시지 저장 후 브로드캐스트하여
데이터 정합성 보장. 낙관적 UI로 체감 속도를 확보하되,
UUID 기반 Dedup으로 중복·순서 문제 방지.

```
// 수정 전
const optimisticIdx = prev.findIndex(
  m => m.clientUuid === clientUuid && m.status === 'SENDING'
  // SENDING일 때만 매칭
);
```

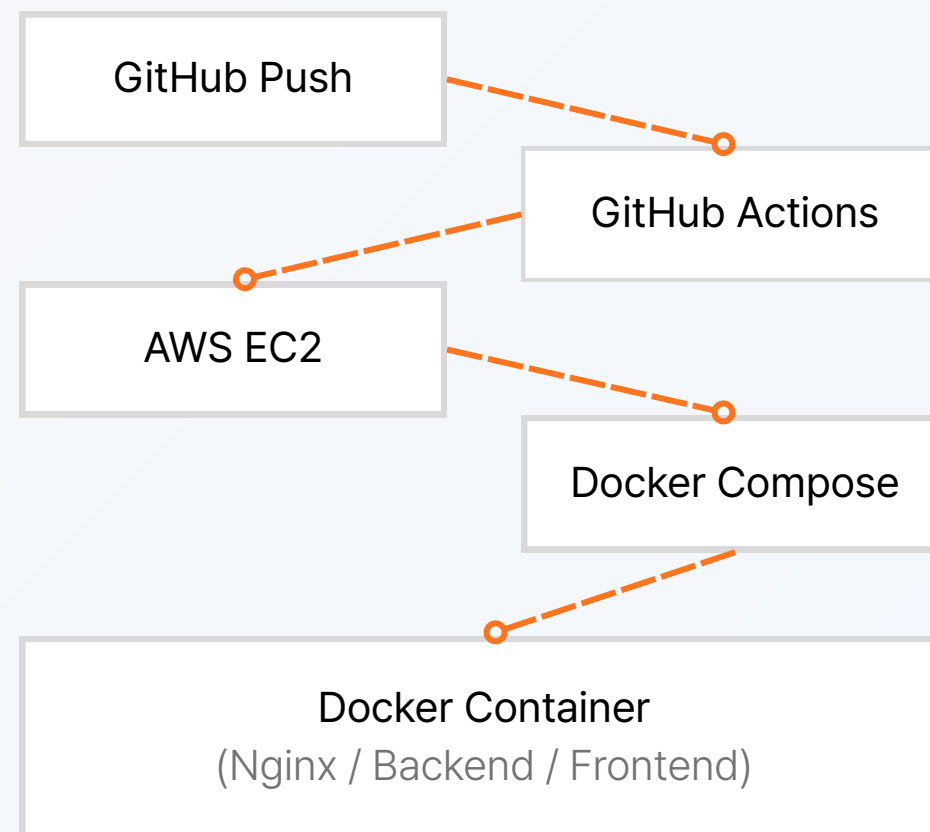
```
// 수정 후 (최종 코드)
const optimisticIdx = prev.findIndex(
  m => m.clientUuid === clientUuid
  // 상태 무관하게 같은 UUID면 매칭
);
```

```
// 수정 전
setMessages(prev => [...prev, optimisticMsg]);
receivedUuids.current.add(clientUuid); // 삭제
scrollToBottom();
```

```
// 수정 후 (최종 코드)
setMessages(prev => [...prev, optimisticMsg]);
scrollToBottom(); // receivedUuids 등록 코드 삭제
```

결제 원자성 보장과 DB 구조 개선,
배포 자동화 및 실시간 통신 안정화를
핵심 전략으로 수립했습니다.

배포 과정



Nginx 프록시 설정 보강 및 배포 자동화

WebSocket 연결 실패 SSE 알림 지연

WebSocket 핸드셰이크 헤더 누락과 SSE 응답 버퍼링을 Nginx 설정으로 해결. GitHub Actions와 Docker Compose를 연동해 push 시 자동 배포 되는 파이프라인 구축.

자체 Oracle DB 서버 구축으로 인프라 통제권 확보

자체 운영 DB 구축

외부 의존 구조를 탈피해 AWS EC2에 Docker 기반 Oracle XE 18c 컨테이너를 직접 구성. DB 스냅샷을 분석해 DDL을 재구성하여 데이터 구조를 복원. GitHub Secrets와 연동된 배포 파이프라인에 자체 DB 서버를 연결해 인프라 전체를 편입.

결제 무결성 보장 및 DB 정규화

통합 결제 내역

전체 상황

상황 관리

통합 결제 내역

상대 승인 시작

시작일 2026-05-04 00 시

종료일 2026-05-11 23 시

1주

주문 정보

주문자 이름

주문자 이름 검색

거래 번호

카드사/결제대행사/PG/결제취소 승인 번호

승인 번호

카드사/결제대행사/PG/결제취소 승인 번호

최종 결제상태

현재 상태 3

결제완료 3

결제실패 0

입금계정 0

승인대기 0

당월 2

입력 일련 번호 0

엑셀 다운로드

상태	결제상태	승인 시작	고객사 거래번호	결제대행사	주문명	주문자명	결제수단	결제 금액	전체 취소 금액	부분 취소 금액	결제 상태	결제 모드	취소
→	결제완료	2026-05-09 14:33:49	charge_8_2026050914...	KD라이니스	포인트 충전	-	카드결제	1,000 KRW	0 KRW	0 KRW	-	테스트 일반	취소
→	결제완료	2026-05-08 20:39:50	charge_8_2026050820...	KD라이니스	포인트 충전	-	카드결제	1,000 KRW	0 KRW	0 KRW	-	테스트 일반	취소
→	결제완료	2026-05-07 20:30:08	charge_8_2026050720...	KD라이니스	포인트 충전	-	카드결제	1,000 KRW	0 KRW	0 KRW	-	테스트 일반	취소

표 상황

페이지 당 행

10

1 / 1

1

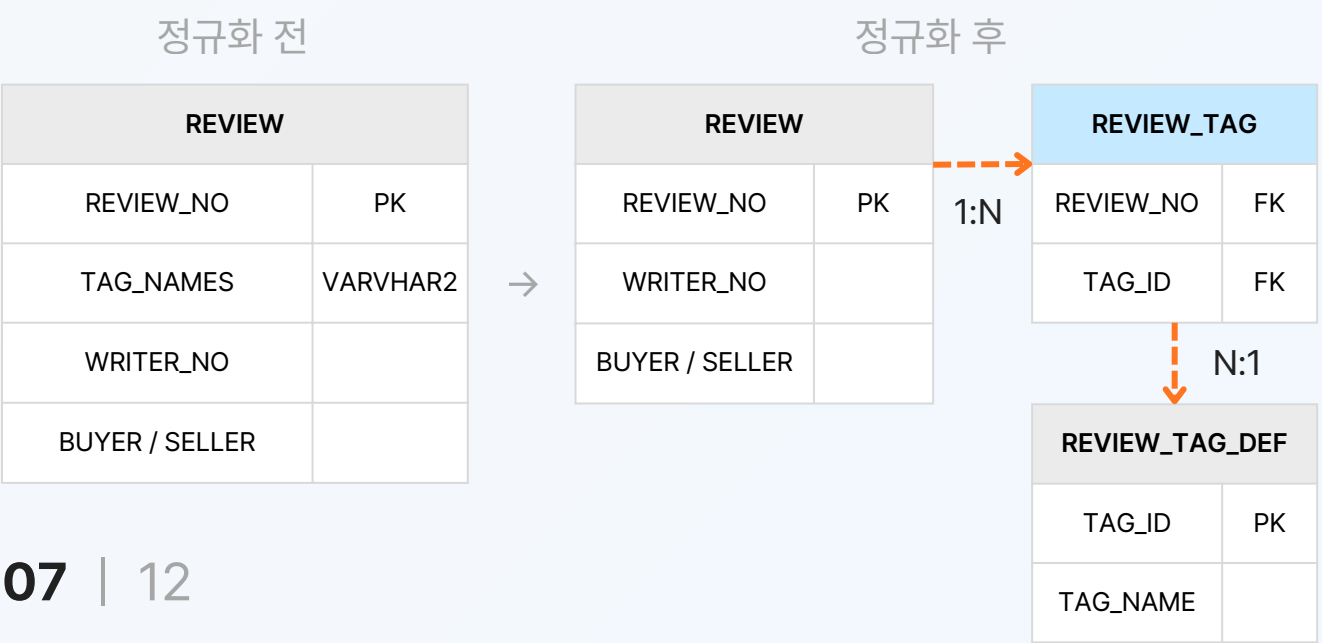
20

PortOne API 연동 → 3단계 단일 트랜잭션 설계

PortOne 결제 요청 → 금액 검증 → 포인트 적립을 단일 트랜잭션으로 묶어 세 단계 중 하나라도 실패하면 전체 롤백. 서버에서 최종 확정된 잔액만 응답으로 내려 화면에 표시.

결과 결제 데이터 정합성 100% 보장. 부분 실패 상황 완전 차단.

정규화 과정

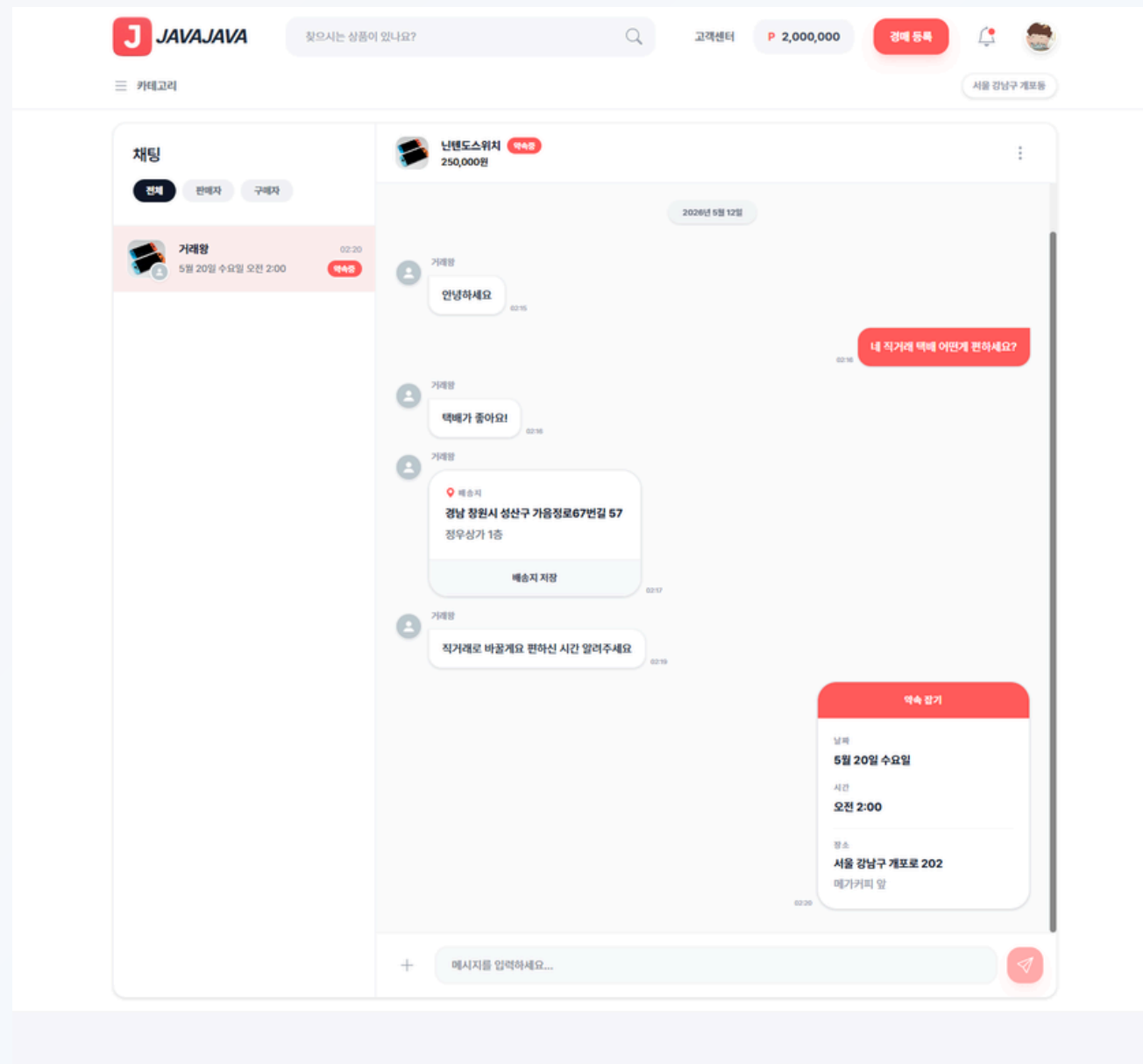


거래 후기 DB 정규화 → 다대다 태그 매핑 분리

태그 정의 테이블과 리뷰-태그 연결 테이블로 분리해 정규화 수행. 관리자 닉네임은 저장 제거 후 조회 시점 JOIN으로 대체. 엔티티부터 Controller까지 전 계층 수정.

결과 역할별 태그 집계 기능 구현 및 담당자 정보 변경 시에도 항상 최신값 보장.

실시간 1:1 채팅 시스템 구현



낙관적 UI 정합성 → UUID Dedup 등록 시점 교정

UUID 등록 시점을 발신 시점에서 서버 응답 수신 시점으로 이동. 타임아웃으로 실패 처리된 후에도 뒤늦게 도착한 서버 응답을 정상 반영하도록 보강.

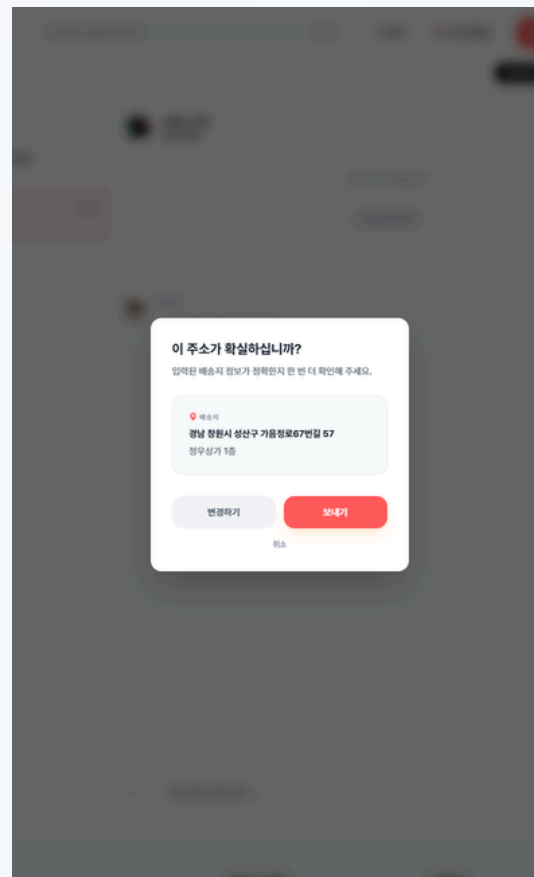
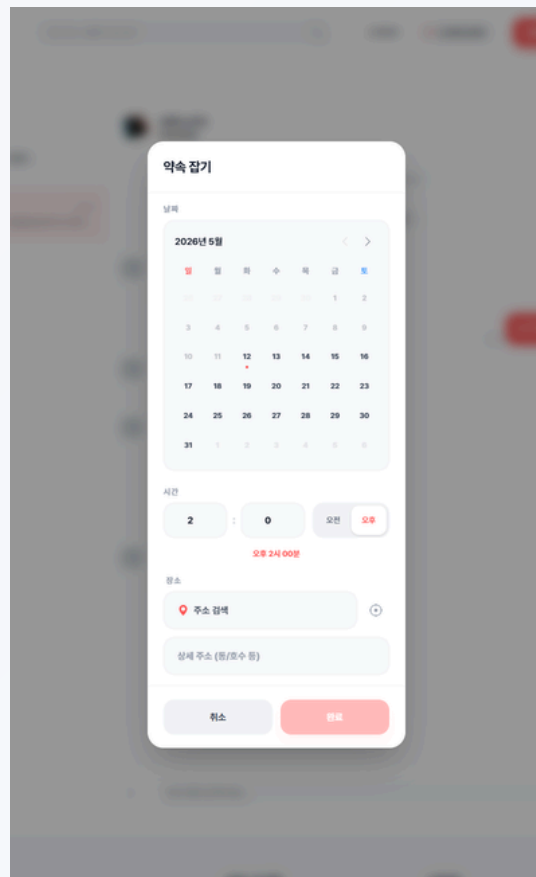
결과 실시간 메시지 확정 보장. 타임아웃 경합 해소 및 낙관적 UI 제공.

채팅방 목록 N+1 최적화 → 단일 쿼리 조회 전환

Oracle ROW_NUMBER() 윈도우 함수와 서브쿼리를 활용해 방 정보 + 최근 메시지 + 안 읽은 수 + 상대방 프로필을 단일 네이티브 쿼리로 조회. 메시지 내역은 번호 기반 커서 페이징으로 무한 스크롤 구현.

결과 채팅방 목록 조회 쿼리 N+1회 → 1회로 단축.

실시간 1:1 채팅 시스템 구현

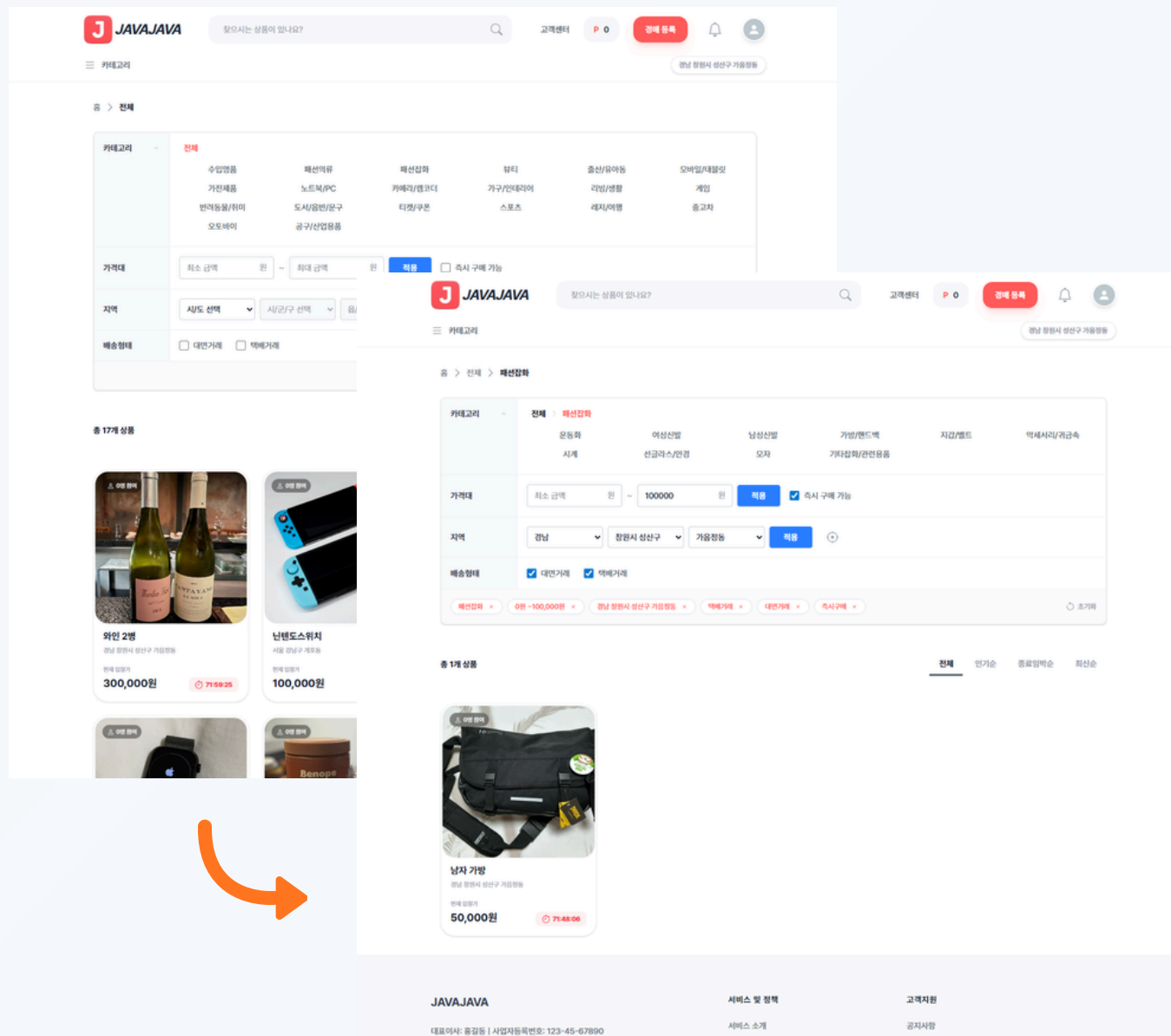


WebSocket+STOMP 아키텍처 → Save-then-Broadcast 패턴

WebSocket으로 양방향 연결을 유지하고,
STOMP 프로토콜로 채팅방별 구독/발행을 관리.
메시지 수신 시 DB에 먼저 저장한 후 브로드캐스트하는
Save-then-Broadcast 패턴을 적용해 저장 실패 시 유령 메시지 발생을 차단.
STOMP 연결 시 JWT 검증, 채널 구독 시 참여자 인가 처리를 적용해 보안 확보.

결과 텍스트 · 이미지 · 배송지 · 약속잡기까지 거래에 필요한 모든 소통을
채팅 하나로 처리. 이미지는 REST 업로드 후 URL을
STOMP에 실어 보내는 2단계 전송으로 메시지 브로커 부하 방지.

쿼리 최적화 및 인프라 구축



카테고리 필터링 → 범위 조건 단일 쿼리 전환

카테고리 ID가 대분류 × 100 = 중분류, 중분류 × 100 = 소분류 규칙을 따른다는 점을 활용해 범위 조건 단일 쿼리로 대체.

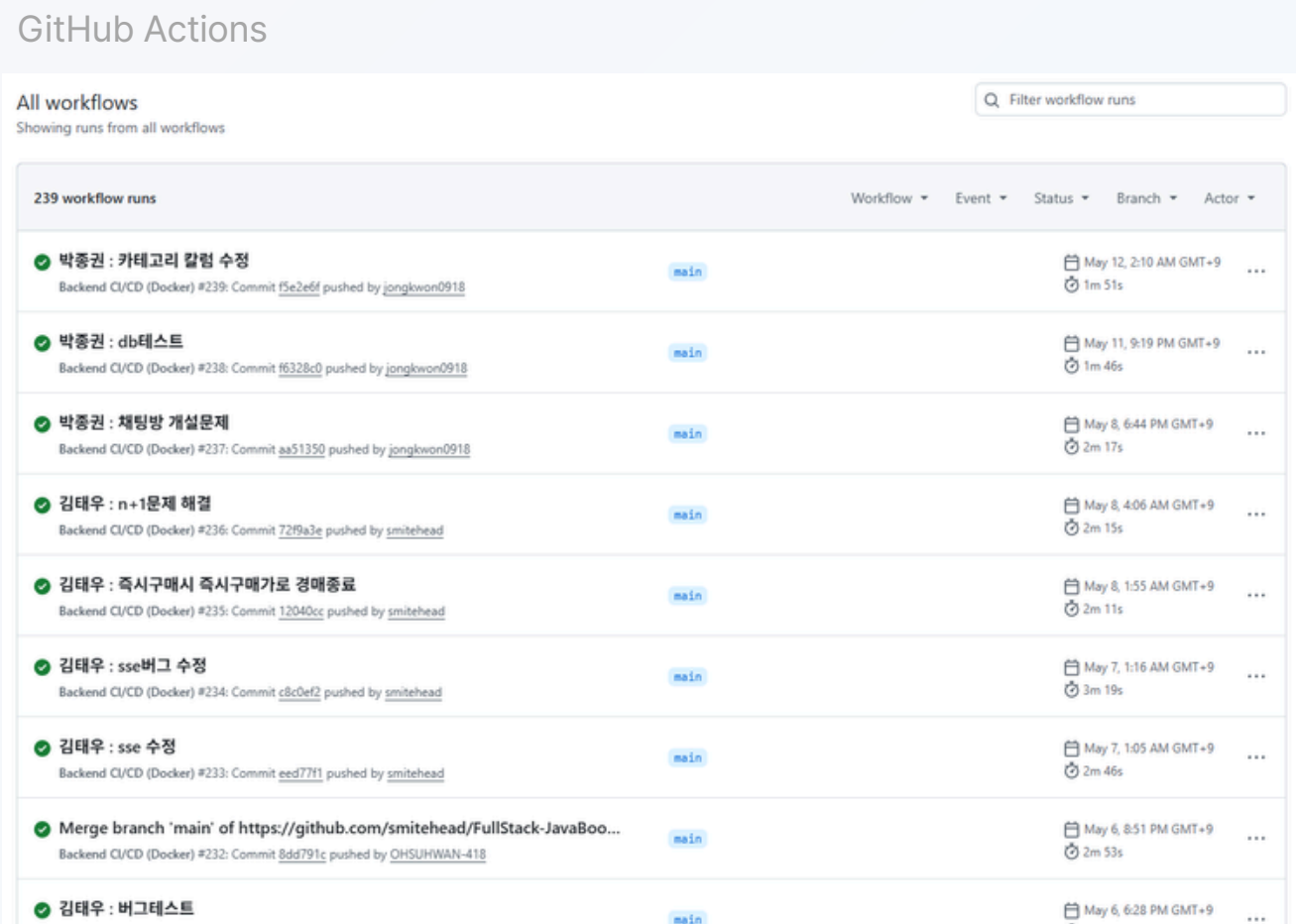
결과 DB 조회 횟수 4회 → 1회로 단축(75% 감소).

Oracle DB 서버 자체 구축 → 외부 의존 탈피 및 인프라 내재화

스왑 메모리 2GB를 추가해 메모리 제약을 해소한 뒤, Docker로 Oracle XE 컨테이너를 구동. DB 스냅샷 JSON을 분석해 32개 테이블·30개 시퀀스 DDL을 재구성하고 PDB 환경에 적용.

결과 기존 배포 파이프라인 변경 없이 DB 서버 전환 완료.

쿼리 최적화 및 인프라 구축



CI/CD 파이프라인 구축

GitHub Actions에서 빌드 후 SCP로 AWS EC2에 전송, SSH로 Docker Compose를 실행하는 자동 배포 파이프라인 구성. master 브랜치 push 시 프론트엔드(npm 빌드)와 백엔드(Gradle 빌드)가 순차적으로 자동 실행.

결과 배포 프로세스 전체 자동화. 휴먼 에러 제거.

Nginx 실시간 통신 안정화

WebSocket은 Nginx에 Upgrade / Connection 헤더를 명시적으로 추가. SSE는 proxy_buffering off 및 proxy_read_timeout 86400s 설정으로 버퍼링 비활성화.

결과 로컬과 배포 환경 동일한 실시간 통신 동작 보장.

결제 정합성과 DB 조회 효율을 확보하고
안정적인 배포 환경과 DB 독립화를
달성했습니다.

✨ 아키텍처 개선 인사이트

- 외부 API와 내부 상태 변경은 단일 트랜잭션으로 묶어야 정합성이 보장됨을 체감.
- 비동기 흐름에서 데이터 등록·확인 시점 제어가 정합성의 핵심임을 수립.
- 로컬과 배포 환경의 차이는 인프라 레이어에서 먼저 확인해야 한다는 디버깅 원칙 정립.

	Before	After	성과
결제 데이터 정합성	부분 실패 시 데이터 불일치	단일 트랜잭션 롤백 보장	정합성 100%
카테고리 DB 조회 횟수	최대 4회 재귀 조회	1회 범위 조건 단일 쿼리	75% 감소
리뷰 태그 구조	단일 컬럼 문자열 저장	정규화된 다대다 매핑	집계 기능 구현 가능
채팅방 목록 조회	채팅방 수(N)만큼 추가 쿼리	ROW_NUMBER() 단일 쿼리	N+1 → 1회
낙관적 UI 메시지 확정	발신자 화면 재전송 무한 반복	UUID 등록 시점 교정	즉시 확정 보장
배포 프로세스	수동 빌드 및 서버 전송	push 시 전체 자동화	배포 자동화 완성
배포 환경 실시간 통신	WebSocket 405 / SSE 지연	Nginx 설정 보완으로 해결	로컬·배포 동일 동작
DB 인프라 통제권	외부 공유 서버 의존	자체 Oracle DB 구축	인프라 완전 내재화



AI 모델 헬스케어 앱 서비스

Summary

- 음식 사진 한 장 또는 성분표로 칼로리와 영양소를 즉시 분석하고 개인 맞춤형 피드백을 제공하는 Android 헬스케어 앱의 Firebase 백엔드 로직과 데이터 처리 파이프라인을 담당했습니다.

Action

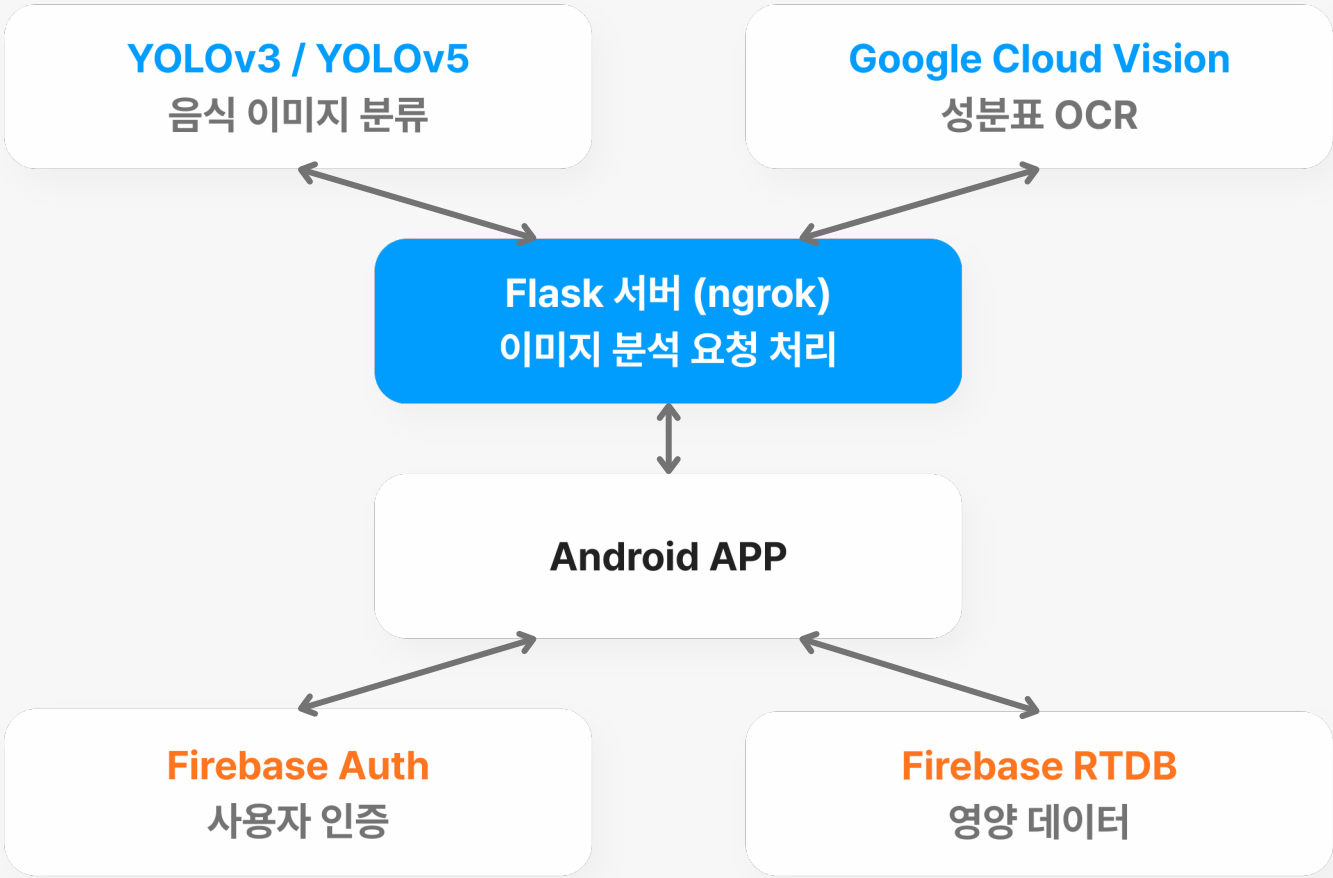
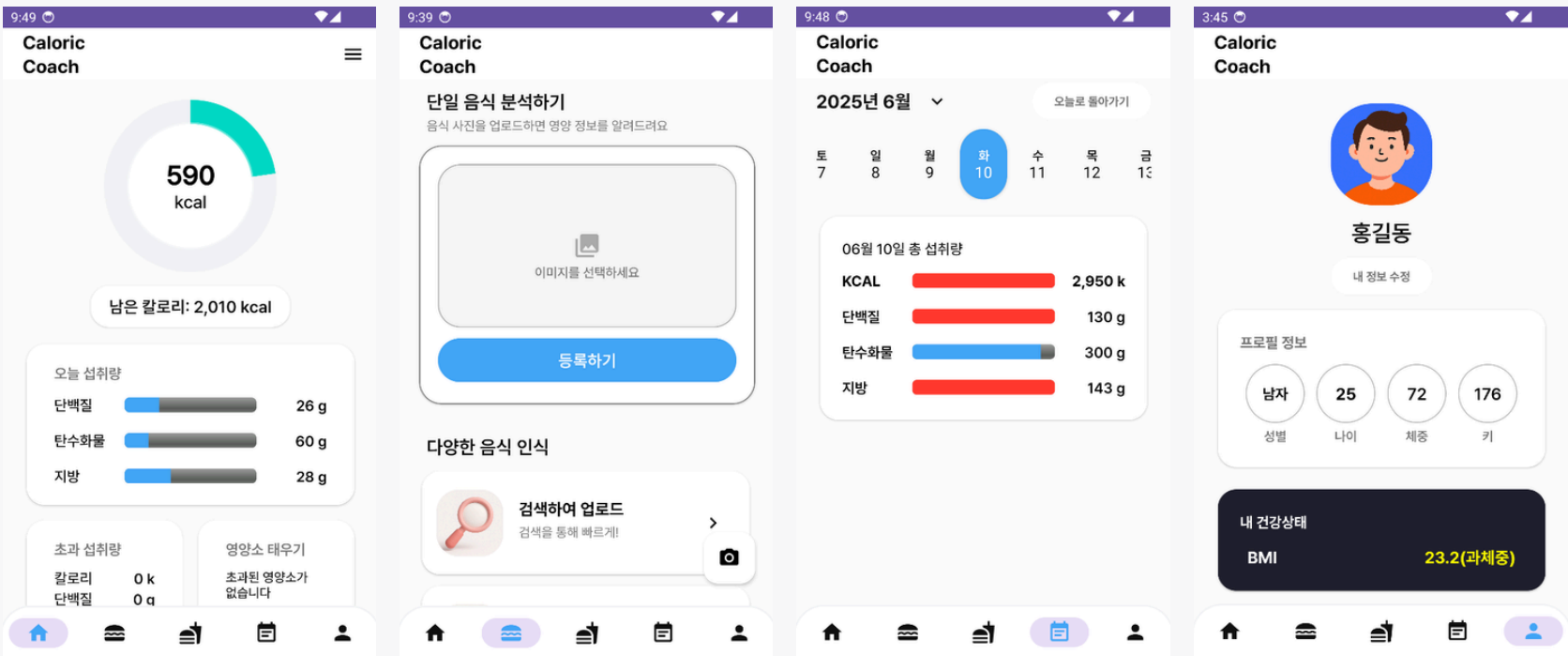
- Firebase Authentication 기반 회원가입·로그인 및 콜백 안정화
- Flask 서버 JSON 응답 파싱 후 Firebase Realtime Database 저장 로직 설계
- 섭취량(g) 기반 비례 영양소 계산 및 UID/날짜 경로 저장 구조 구현
- 홈 화면 당일 섭취 집계 · 초과 감지 비즈니스 로직 구현

Period

2024.11 - 2025.06

Team

3명 (BE·FE 3)



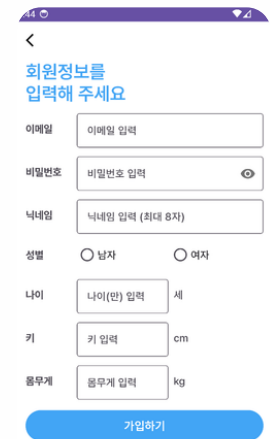
인증 콜백 결함과 서버-클라이언트 데이터 불일치, 집계 로직 부재를 파악했습니다.

Firestore 인증 콜백 누락

비정상 종료

회원가입 · 로그인 후 Firebase Auth 결과를 기다리지 않고
다음 화면으로 전환하거나, 잘못된 이메일 형식 입력 시 앱이
강제 종료되는 현상 발생.

→ 성공 · 실패 콜백을 처리하지 않고 동기식으로 동작한다고 가정한 것이 원인.

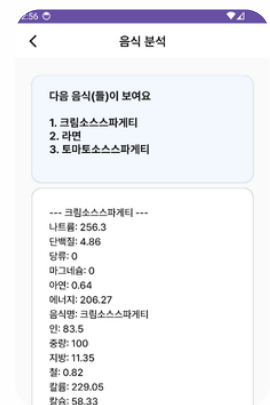


서버 응답 JSON 구조 불일치

NullPointerException

Flask 서버가 반환하는 음식 분류 JSON의 키 구조와 앱에서
파싱하는 구조가 달라 NullPointerException 발생.
다중 인식 결과(최대 15개) 배열 순회 로직도 미구현 상태.

→ 서버-클라이언트 간 데이터 계약(JSON 스키마)을 사전에
명세하지 않은 것이 원인.



인증 콜백 결함과 서버-클라이언트 데이터 불일치, 집계 로직 부재를 파악했습니다.

당일 섭취 집계 로직 부재

홈 화면 정적 상태

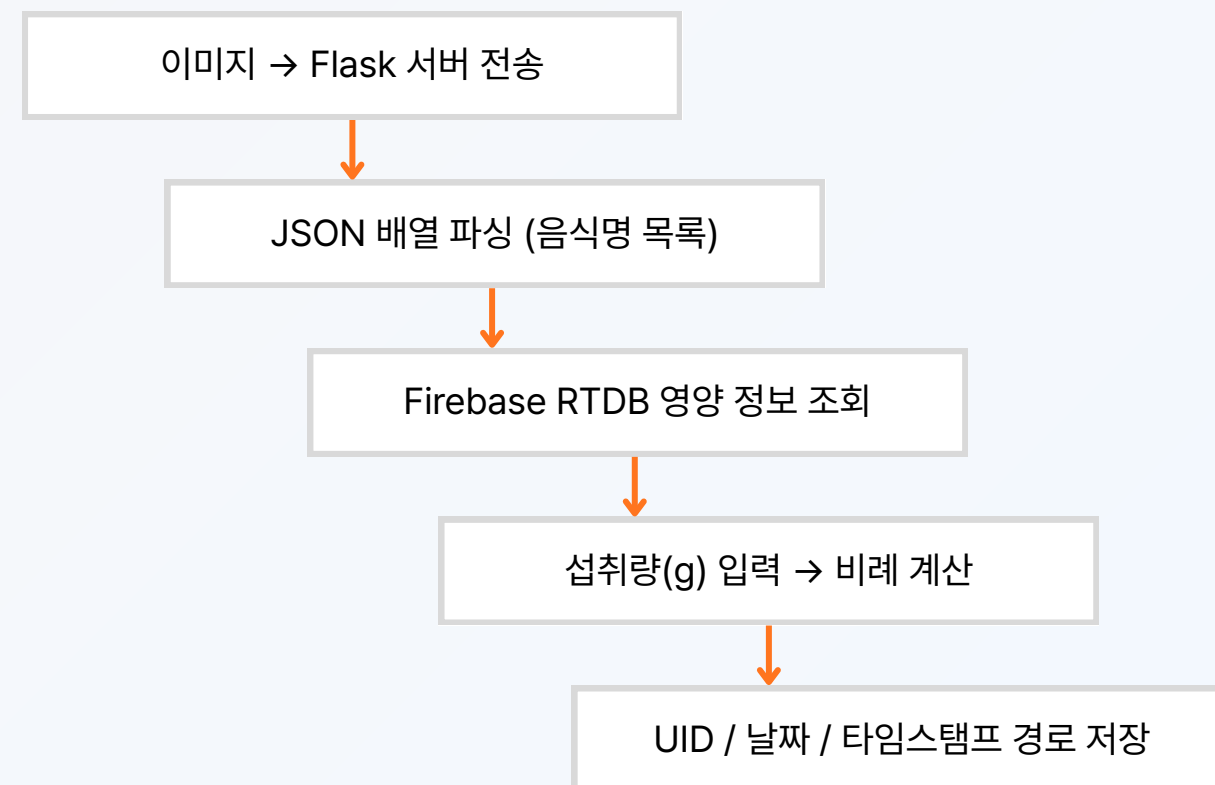
Firestore에 음식 데이터를 저장하지만 이를 집계해
남은 칼로리 · 초과 영양소를 표시하는 비즈니스 로직이 없는 상태.



→ 단순 CRUD만 구현된 상태로, 저장된 데이터를 실제 서비스 로직에
활용하지 못한 설계 결함이 원인.

인증 흐름 완전 처리,
데이터 파이프라인 정합성,
집계 로직 설계를 핵심 전략으로 수립했습니다.

저장 데이터 플로우



Firestore Auth 비동기 흐름 완전 처리

인증 결함

Auth 결과를 콜백으로만 처리하도록 구조 변경. 성공 시 UID 기반 RTDB 저장 후 화면 전환, 실패 시 에러 코드별 메시지를 UI에 반영하는 분기 처리 설계. Firestore 호출 전 단계에서 입력 유효성 검사를 선행.

서버 응답 → Firestore 저장 파이프라인 설계

JSON 불일치 · 데이터 플로우

Flask 응답 구조를 분석해 앱 파싱 키를 일치시키고, 결과 배열 순회 → Firestore 영양 정보 조회 → 섭취량 비레 계산 → UID/닉네임 경로 저장까지 단일 파이프라인으로 설계.

**인증 흐름 완전 처리,
데이터 파이프라인 정합성,
집계 로직 설계를 핵심 전략으로 수립했습니다.**

홈 화면 집계 → 피드백 로직 설계

비즈니스 로직 부재

Firebase 당일 데이터 전체 조회 후 칼로리 · 영양소 합산,
권장값과 비교해 초과 여부를 판정하는 구조를
하나의 데이터 조회 로직 안에서 설계.

Firebase 인증 안정화

9:44

Caloric Coach

이메일 입력

비밀번호 입력

로그인

또는

회원가입

9:44

<

회원정보를 입력해 주세요

이메일

이메일 입력

비밀번호

비밀번호 입력

닉네임

닉네임 입력 (최대 8자)

성별

☐ 남자

☐ 여자

나이

나이(만) 입력

세

키

키 입력

cm

몸무게

몸무게 입력

kg

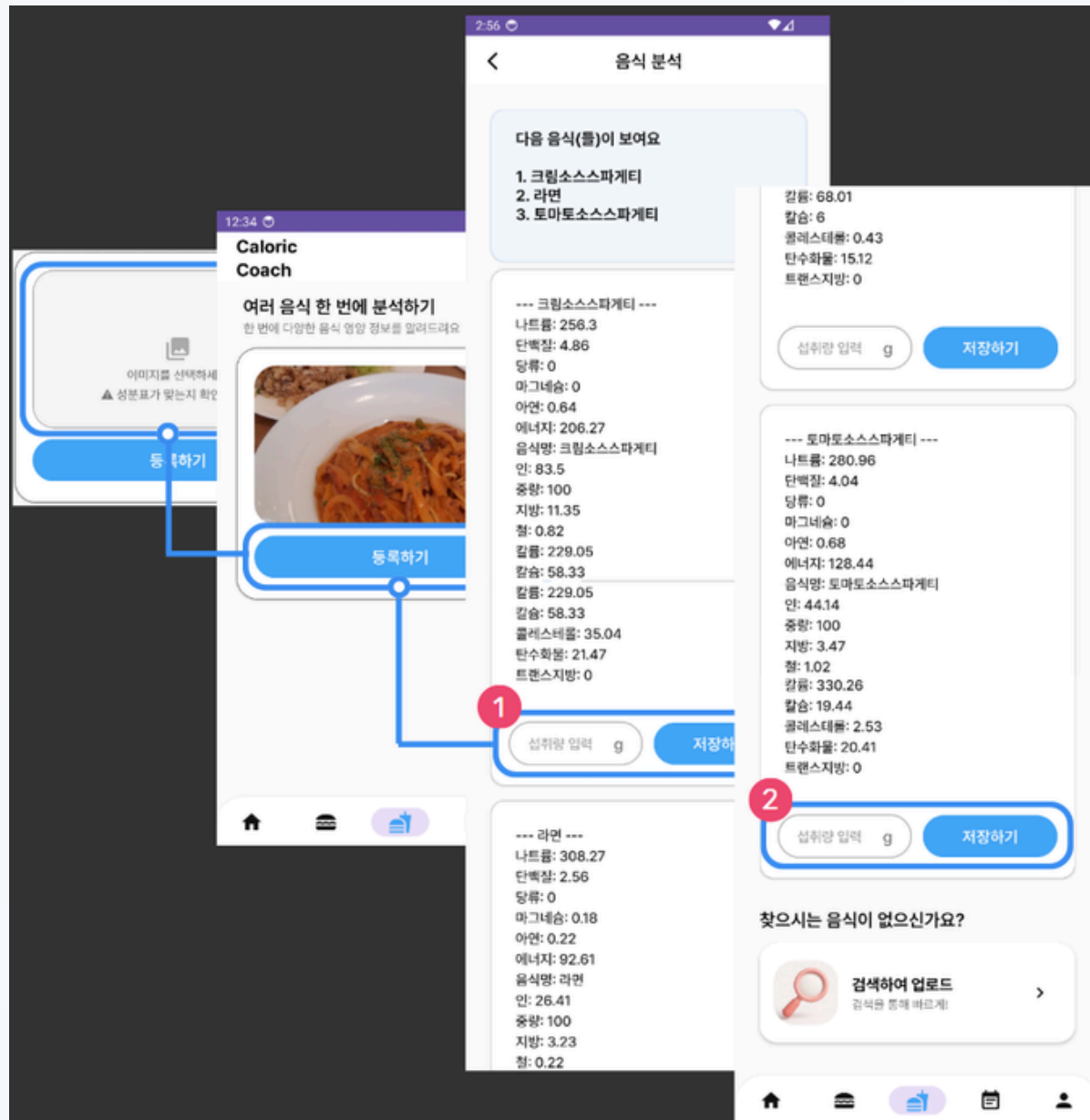
가입하기

비동기 콜백 완전 처리

Firebase Auth 콜백을 성공·실패로 분기 처리해 비동기 타이밍 문제를 구조적으로 해결. 성공 시에만 UID 기반 RTDB 저장 후 화면 전환하고, Firebase 호출 이전에 이메일 형식·비밀번호 길이·닉네임 8자 제한을 선검사해 잘못된 입력이 Auth로 넘어가는 경우를 원천 차단.

결과 인증 흐름 정상화 · 비정상 종료 0건

JSON 파싱 & Firebase 저장



응답 구조 일치 및 다중 결과 처리

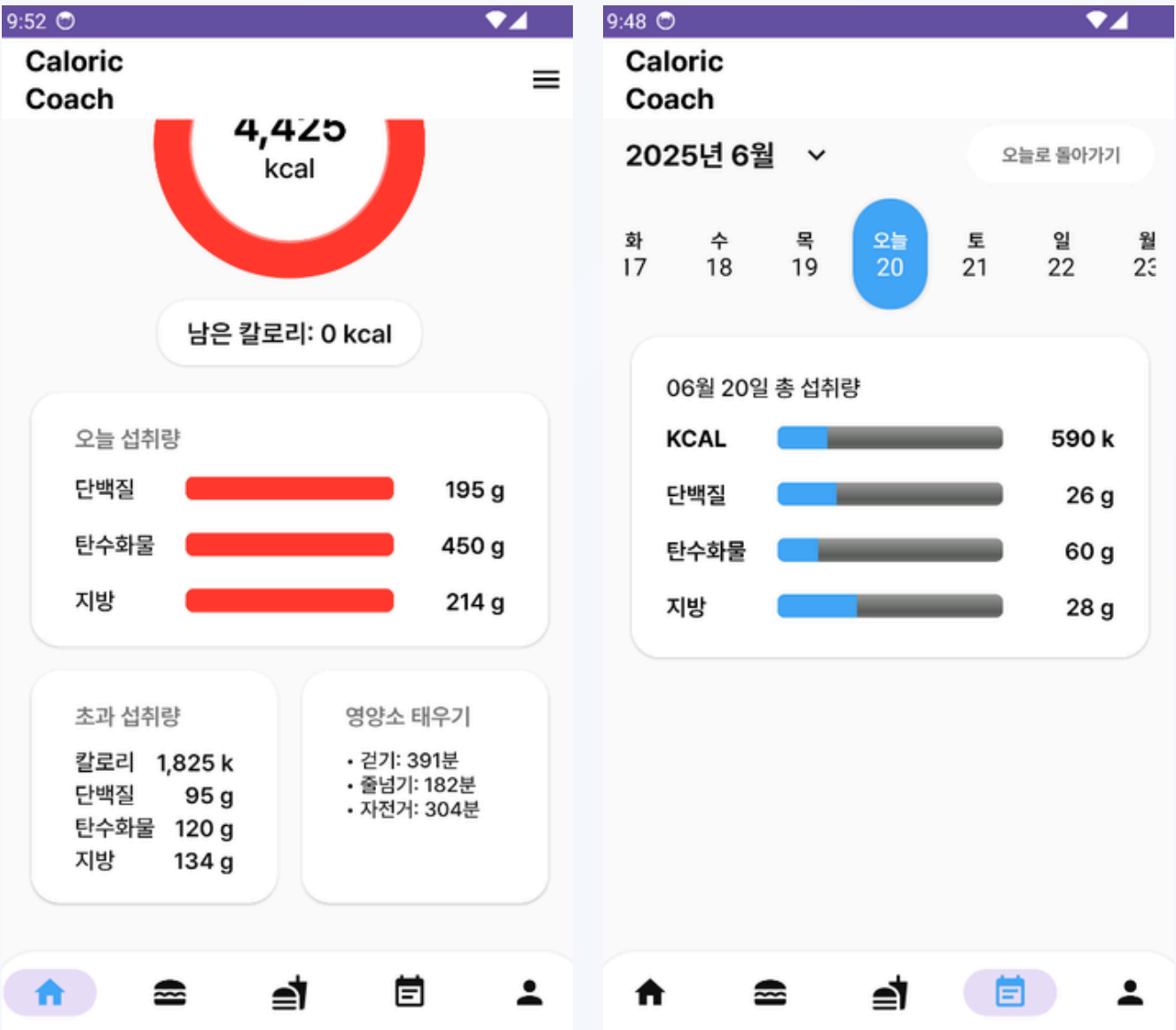
Flask 서버 응답 JSON 배열(최대 15개)을 순회하며 음식명별로 Firebase에서 영양 정보를 조회.

사용자가 입력한 섭취량(g)에 100g 기준 영양값을 비례 계산한 뒤 UID/날짜/타임스탬프 경로로 저장.

성분표 OCR 결과도 동일한 파이프라인으로 처리해 데이터 저장 흐름을 단일 구조로 통일.

결과 파싱 오류 0건 · 다중 음식 저장 정상 처리

당일 집계 & 운동 추천



당일 집계 & 초과 감지

오늘 날짜 경로의 Firebase 데이터를 단일 쿼리로 전체 조회한 뒤
칼로리 · 단백질 · 탄수화물 · 지방을 합산.
결과는 PieChart와 ProgressBar로 실시간 시각화함.

결과 초과 즉시 감지·실시간 시각화 완료

인증 안정화, 데이터 정합성,
실시간 집계 및 초과 감지를
달성했습니다.

✨ 아키텍처 개선 인사이트

- 비동기 콜백은 성공 · 실패를 모두 명시적으로 처리해야 앱 안정성이 보장됨을 확인.
- 서버-클라이언트 데이터 구조를 사전에 맞추지 않으면 통합 단계에서 오류가 발생함을 학습.
- 저장 기능만으로는 반쪽짜리 백엔드이며, 데이터를 비즈니스 로직에 연결해야 실제 서비스가 된다는 원칙 정립

	Before	After	성과
Firebase 인증	콜백 누락 · 비정상 종료	비동기 콜백 완전 처리	인증 안정화 완료
JSON 파싱	키 불일치 · NPE 발생	구조 일치 + 배열 순회 로직 추가	파싱 오류 0건
영양 데이터 저장	단순 저장만 가능	섭취량 비례 계산 → UID/날짜 경로 저장	데이터 정합성 확보
홈 화면 집계	정적 화면 · 집계 없음	당일 전체 조회 후 실시간 합산	실시간 집계 완료
초과 감지	미구현	당일 합산 후 권장값 초과 여부 즉시 판정	초과 알림 기능 완성

WEB BACKEND

PORTFOLIO END

박종권

- 안정적인 트랜잭션 처리와 서버 부하 최적화 역량 보유
- 에러 로그 및 트러블슈팅 과정 문서화